



ModelForge

Version a model. Release progressively. Preserve the stable service.

ModelForge is a model registry, deployment control plane and inference platform. Browser, CLI and API clients use the same operations to register immutable model versions, select a runtime, send a weighted share of requests to a canary and promote, abort or roll back a release.

The problem

Shipping a model is a release-management problem as well as an inference problem. An artifact must retain a verifiable identity, a candidate needs bounded exposure, and a failed new version must preserve the stable target.

What you can do

Immutable model versions Registered artifacts have SHA-256 identities, verified before execution, with local or S3-compatible storage behind one contract.	Progressive release controls Weighted canaries receive controlled traffic. Promotion, abort and rollback preserve validated lifecycle transitions and environment targets.
Framework-independent execution Built-in, PyTorch and ONNX runtimes share an interface. An external HTTP contract supports the reference Go runtime and future plugins.	Runtime failure containment Health inspection, bounded retries, circuit breakers, metrics and stable fallback govern execution at the runtime boundary.

How it works

01 / Register Create a workspace-scoped model and immutable version, then upload and verify its artifact.	02 / Deploy An environment resolves its authoritative stable target through the selected runtime.
03 / Canary A candidate receives a weighted lane while stable continues serving; responses identify deployment, version and lane.	04 / Promote or recover Promote a healthy candidate, roll back a release or abort a failed canary while preserving stable.

TECHNOLOGY / Python · FastAPI · SQLAlchemy · MySQL · Go · PyTorch · ONNX · Docker · Terraform · Prometheus · Amazon Web Services

Decisions that shape the system

Control plane and execution are separate

The Python/FastAPI control plane owns identity, state, traffic and tenancy. Runtimes own framework-specific loading and prediction, so a new framework does not require serving-path branches.

Atomic release invariants

A canary does not become stable until promotion succeeds. Promotion and rollback update release state and environment targets transactionally.

Workspace boundaries throughout

Models, artifacts, deployments, canaries and keys are workspace-scoped. Browser sessions use OIDC/PKCE; automation uses hashed workspace API keys.

Observed validation

The recorded CI checkpoint passed 164 tests with 3 skips. The deployment validation scenario serves a baseline, observes weighted canary traffic, promotes v2, restores v1, detects an intentionally regressed v3 and aborts it while v1 remains authoritative.

Scope of this evidence

ModelForge is not publicly hosted. The console preview uses example data and is illustrative. AWS/Terraform configuration is a deployment reference, not proof of live AWS operation. Model caches and runtime client state are currently process-local.

Explore the implementation

Source code and local setup

github.com/michaelbawuah/ModelForge

Architecture and invariants

github.com/michaelbawuah/ModelForge/blob/main/docs/architecture.md

Deployment validation scenario

github.com/michaelbawuah/ModelForge/blob/main/docs/demo-proof.md

External runtime protocol

github.com/michaelbawuah/ModelForge/blob/main/docs/runtime-protocol.md

Engineering guide

github.com/michaelbawuah/ModelForge/blob/main/docs/engineering-guide.md

This brief explains the project using its public source and retained validation records. It is an engineering overview, not a research publication.